stream channel establishes subscriptions using signed stream names, while broadcasts deliver refresh actions through Action Cable.

The application configures publishers and subscribers for the same trip-scoped stream. This supports a framework-mediated Observer classification without claiming that DaPlanStan implements its own observer registry or browser notification runtime.

Counting decision

Count one Trip-Scoped Turbo Refresh Notifications instance across geocoding and inbound-email processing.

Together with Outbound Mail Delivery Logging, there are two source-confirmed Observer instances in the investigations completed so far.

Limitations

Runtime stream delivery, Turbo morphing behavior, permanent-map updates and browser reconnection have not been tested. These remain separate from the static pattern decision.

The original archive's correspondence with the stated Git commit and independent human verification also remain unresolved.

Frontend investigation conclusion: One proposed Observer classification is rejected as targeted DOM event communication. The existing Turbo candidate is confirmed as one framework-mediated Observer collaboration. No additional Observer instance should be created for its repeated appearances across source areas.

We have now completed focused investigations of geocoding, inbound email processing, trip creation and membership, itinerary import and expenses, external integrations and identity, and frontend interactions.
Before freezing the baseline inventory, do one repository-wide sweep to check whether any important pattern collaboration has been missed.
Do not start by trying to increase the number of patterns.
Use the patterns and rejected candidates already established as context, but inspect the remaining application source independently.

Current source-confirmed collaborations are:

1. GRASP Information Expert — Parsed Email to Trip Record Linking
2. GRASP Controller — Inbound Travel Email Intake Coordination
3. GRASP Controller — Trip Creation Coordination
4. GRASP Controller — Trip Membership Management Coordination
5. GRASP Controller — Bulk Itinerary Import Coordination
6. GRASP Controller — OAuth Sign-in and Identity Linking Coordination
7. Observer — Outbound Mail Delivery Logging
8. Observer — Trip-Scoped Turbo Refresh Notifications

Previously rejected examples include Template Method, Strategy and Adapter in geocoding, and Observer for the targeted timeline-to-map DOM events.
For this sweep:

1. Review the application source areas that have not received the same focused treatment yet.
2. Check controllers, models, concerns, jobs, mailboxes, mailers, helpers, initializers and JavaScript for collaborations that involve multiple application participants.
3. Pay attention to creation responsibility, responsibility assignment, object composition, indirection, polymorphic behavior, notification relationships and algorithms that are genuinely interchangeable.
4. Check whether any previously inspected source contains a second independent collaboration that was overlooked.
5. Do not treat Rails inheritance, enums, callbacks, Active Record associations, `create!` calls, background jobs, mailers or Stimulus controllers as patterns by themselves.
6. Do not propose another GRASP Controller merely because an unreviewed controller handles CRUD. It must coordinate a meaningful system event comparable to the already confirmed instances.
7. Do not propose Information Expert merely because a model contains methods. Show why the responsibility belongs with the information held by that object.

8. Do not propose Strategy unless there are actually interchangeable algorithms or implementations selected through a common role.
9. Do not propose Adapter unless there is real interface translation between incompatible collaborators.
10. Do not propose Factory or Creator merely because objects are constructed.
11. Do not propose Observer merely because callbacks or events exist. Establish publisher, subscription/registration and notification behavior.
12. Check carefully for duplicate instances. Reappearance of an already confirmed collaboration in another file or workflow is not a new instance.

For every genuinely new candidate you find:

1. Give the pattern and category.
2. Give a meaningful provisional instance name.
3. Identify the exact participants.
4. Give source paths and reliable line ranges.
5. Explain the collaboration that supports it.
6. Give the strongest false-positive explanation.
7. Compare it with the closest already confirmed or rejected candidate.
8. Leave it UNCONFIRMED.

Also report important source areas you inspected where no additional pattern was found. That is useful evidence of coverage. At the end give me:

1. Source areas checked in this final sweep.
2. Any genuinely new candidates.
3. Previously known collaborations encountered again and deduplicated.
4. Important rejected similarities.
5. Remaining uninspected or ambiguous source areas, if any.
6. Whether the repository is now sufficiently covered to begin the final adversarial review.

Do not generate `patterns.csv`, counts or UML yet.

Worked for 2m 52s